Linux for Data Scientists

Homework

Full name	Vũ Duy Nhất
Student ID	24127095
Class	24KHDL

Instructions

- Complete every question on your OWN computer, in a real terminal (Linux, macOS, or Windows + WSL).
- For each question you must submit TWO things: (1) a screenshot from your machine showing the command AND its output, and (2) a short written explanation of the syntax you used — what each command, flag, and symbol does, in your own words.
- A screenshot with no explanation, or an explanation with no screenshot, earns no marks for that question. The screenshot must show your own environment (your username/prompt visible is ideal).
- Questions in Parts 2–3 must be solved with the shell only — no Python or pandas.
- Submit this document as a Word (.docx) file named yourID_linux_assignment.docx.

The dataset (data/sales_messy.csv) has 9 columns: order_id, date, customer, region, product, category, quantity, unit_price, status.

Part 1 — Filesystem & Project Setup

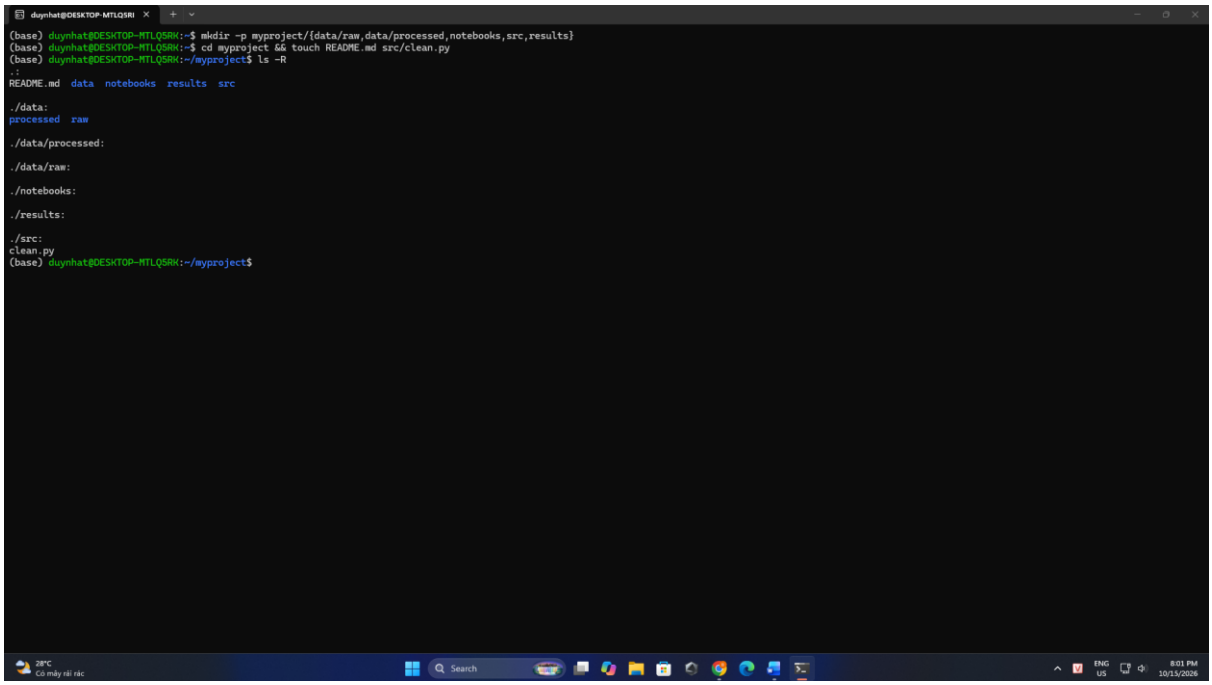
Set up a clean working directory on your own machine before you start.

Q1 · Scaffold a reproducible project

Create the project layout below on your computer, then list it to show the tree. Screenshot the result. In your explanation, say what mkdir -p does and what the { … } brace expansion does.

```
mkdir -p myproject/{data/raw,data/processed,notebooks,src,results}
cd myproject && touch README.md src/clean.py
ls -R       # (or: tree)
```

① Screenshot (paste the command and its output from your own terminal):



② Explanation of the syntax you used:

The command *mkdir -p myproject/{data/raw,data/processed,notebooks,src,results}*:

- ***mkdir*** (make directory) is used to create directories.
- With flag ***-p***, mkdir can create any missing parent directories. It also prevents errors when the target directories already exist.
- The ***{… }*** brace expansion allows multiple directory paths to be specified in a single mkdir command, reducing repetition.

The command *cd myproject && touch README.md src/clean.py*:

- ***cd*** (change directory) is used to change the current working directory to the specified directory.
- The ***&&*** is a command list operator allowing the next command to execute if the command before it completes successfully.
- ***touch*** is used to create empty files if they do not already exist. In this example, it creates README.md in myproject and clean.py in myproject/src.

The command *ls -R*:

- **ls** (list) is used to list the content of the specified directory. If no directory is specified, it executes with current working directory.
- The flag ***-R*** allows ls to list recursively the directory, displaying the contents of the specified directory and all its subdirectories.

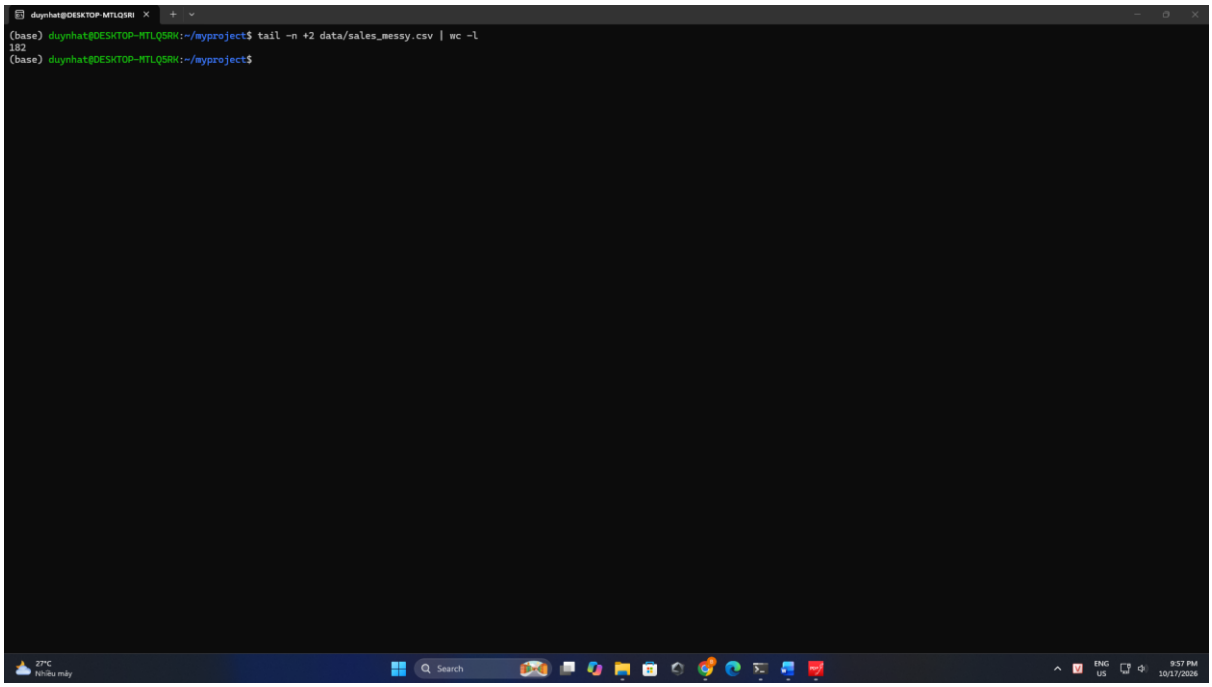
Part 2 — Inspecting Data From the Command Line

All questions use data/sales_messy.csv. Answer using the shell only — no Python.

Q2 · Count the data rows

How many data rows (excluding the header line) are in sales_messy.csv? Show the command and its output, and explain why the answer is not the same as the total line count.

① Screenshot (paste the command and its output from your own terminal):



② Explanation of the syntax you used:

The command *tail -n +2 data/sales_messy.csv | wc -l*:

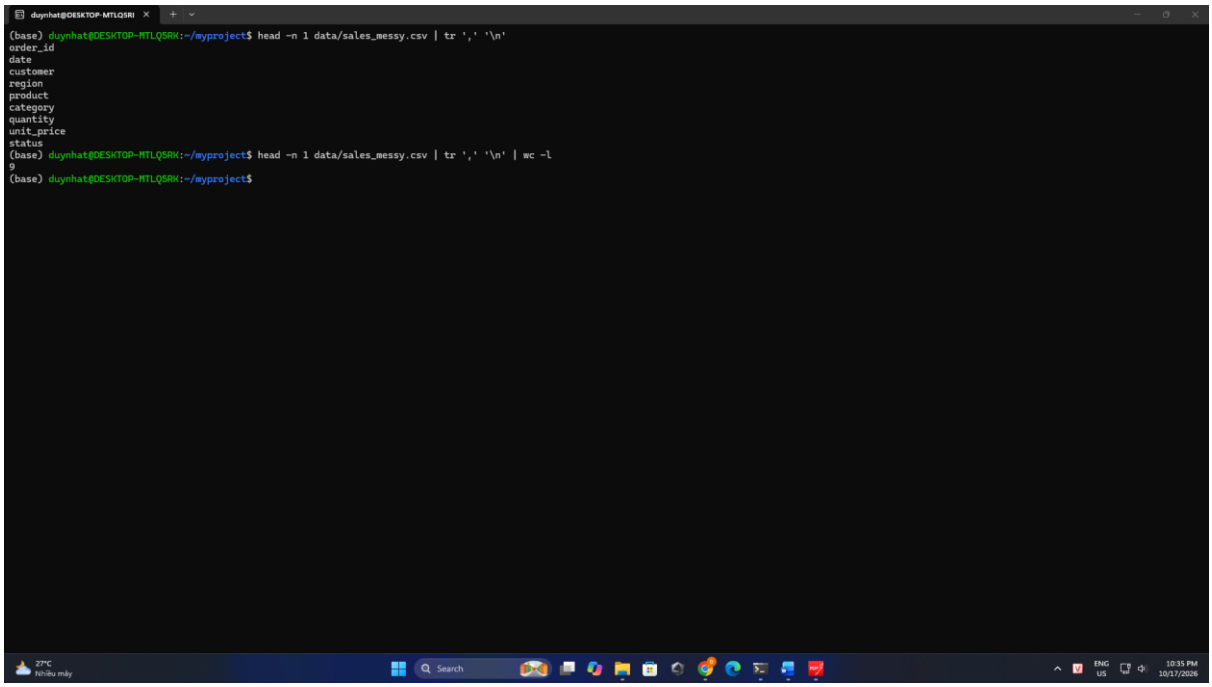
- ***tail*** is used to display the last 10 lines of a file by default. With the ***-n*** flag, it can specify the number of lines to display. In this command, ***-n +2*** tells tail to display the file starting from the second line to the end.
- | is a pipe operator that passes the output of the preceding command as input to the following command.
- ***wc*** with the ***-l*** flag counts the number of lines in the input.

  → The output of the above command is 182 lines while the total lines in that file is 183 lines because it does not count the header line and count from the second line, the output from the command ***tail -n +2***.

Q3 · List the columns

Print the column names, one per line, and report how many columns there are.

① Screenshot (paste the command and its output from your own terminal):



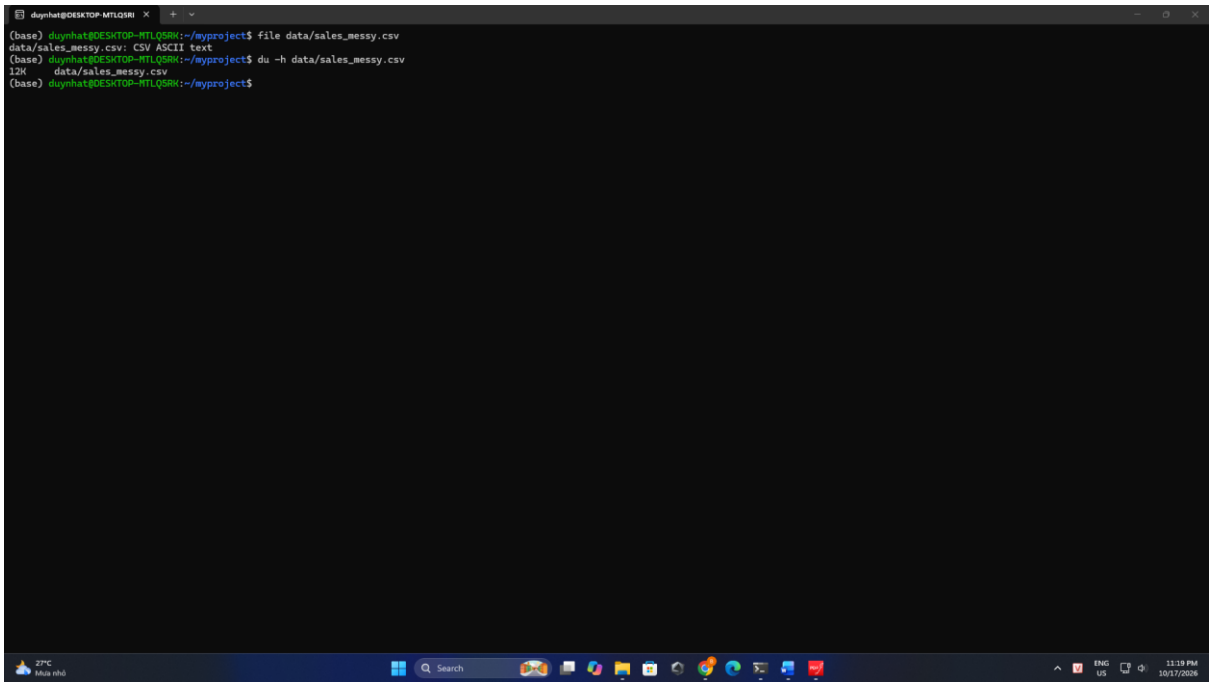
② Explanation of the syntax you used:

- The command ***head -n 1 data/sales_messy.csv*** gets the first line – header, which contains the column headers in the CSV file.
- The output of above command has the format as order_id,date,customer,… and the command ***tr ',' '\n'*** replaces each comma (,) with a newline character (\n) to print one column name per line.
- ***wc*** with the *-l* flag is used to count the number of lines. In this situation, it corresponds with counting the number of columns and there are 9 columns in the CSV file.

Q4 · File type & size

What does the file command report for the CSV, and how large is it on disk (human-readable)?

① Screenshot (paste the command and its output from your own terminal):



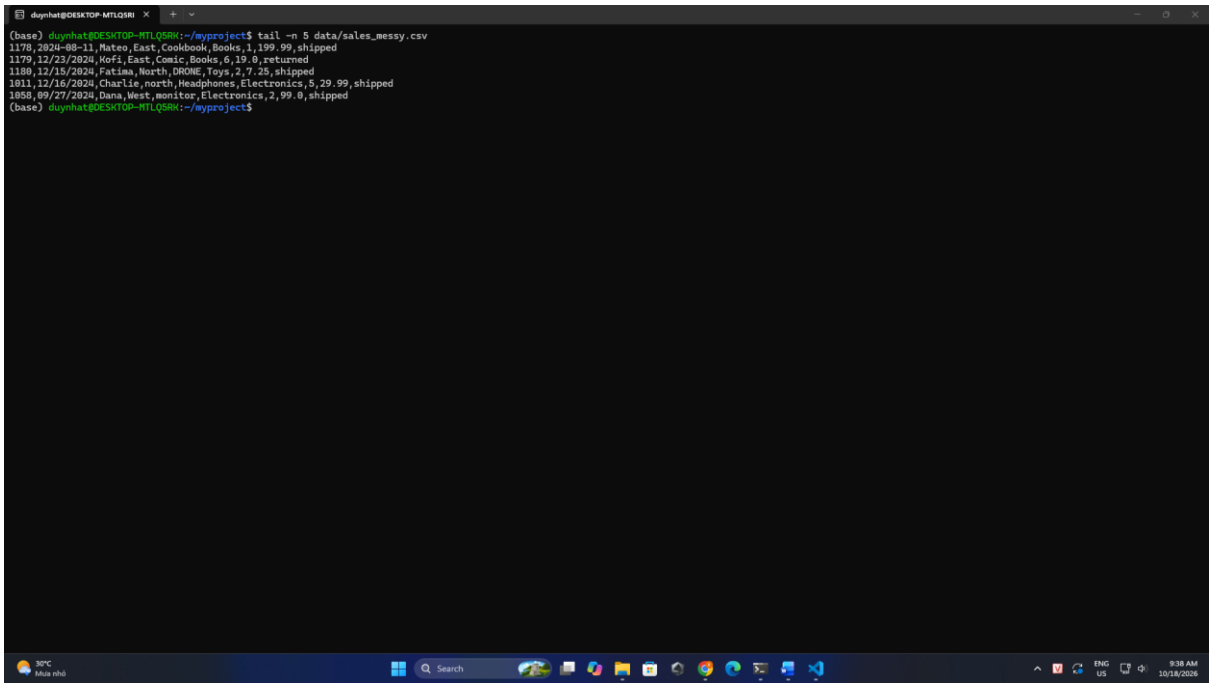
② Explanation of the syntax you used:

- The command ***file data/sales_messy.csv*** identifies the file type and describes its content format. In this case, the output is CSV ASCII text.
- The command ***du -h data/sales_messy.csv*** displays the disk space used by the CSV file. The -h flag displays the size in an easily readable format using units such as KB, MB, and GB. In this case, the file uses 12K (12 Kilobytes) of disk space.

Q5 · Peek at the tail

Show the last 5 rows of the file. Do you notice anything odd about the data? Explain what you see.

① Screenshot (paste the command and its output from your own terminal):



② Explanation of the syntax you used:

The command ***tail -n 5 data/sales_messy.csv*** prints the last 5 lines of the file (-n 5 sets the number of lines).
In last 5 rows, there are some odd about data:

- In the order_id column, the id values are not ascending because of the interruption of id '1011' and '1058'.
- In the date column, the format of date is not unified, the first line has the YYYY-MM-DD format while the others use the MM/DD/YYYY format.
- In the region column, the string 'north' in the fourth line starts with a lowercase letter, whereas the other region values shown start with uppercase letters.
- In the product column, the string 'DRONE' in the third line is fully uppercase and the string 'monitor' in the fifth line is fully lowercase, whereas the other product names only have the first character in uppercase.

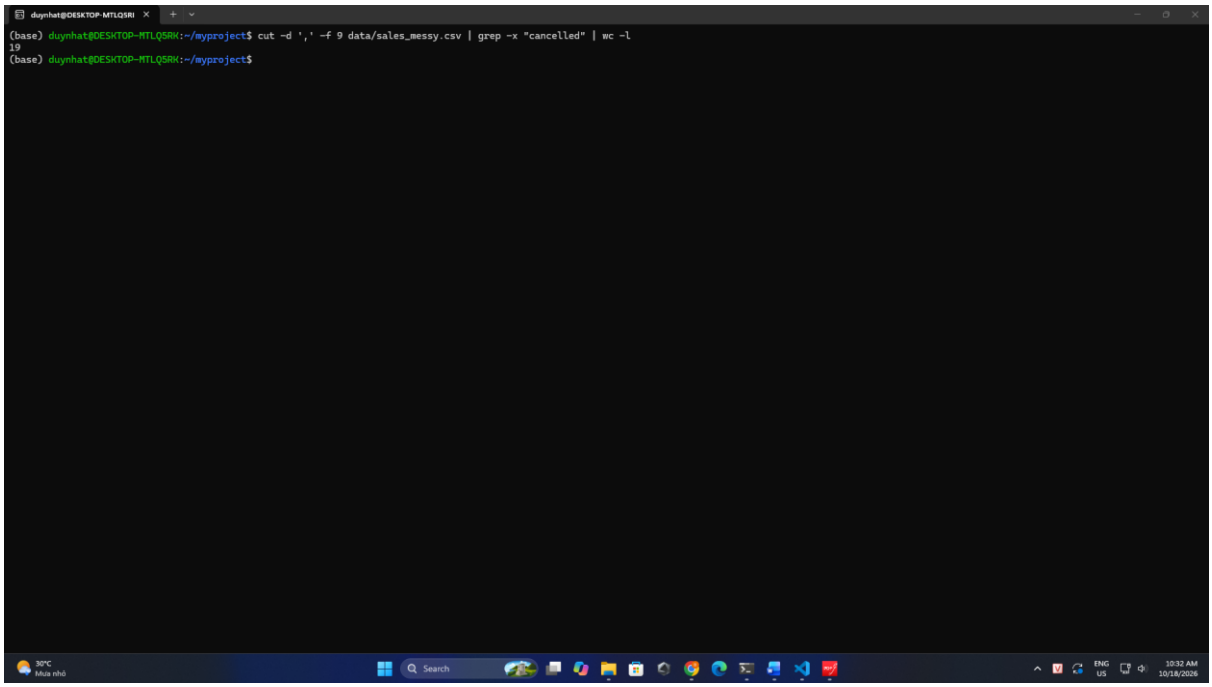
Part 3 — Pipes, Redirection & Text Processing

Build each answer by composing small tools with pipes. No Python.

Q6 · Count cancelled orders

How many orders have the status cancelled?

① Screenshot (paste the command and its output from your own terminal):



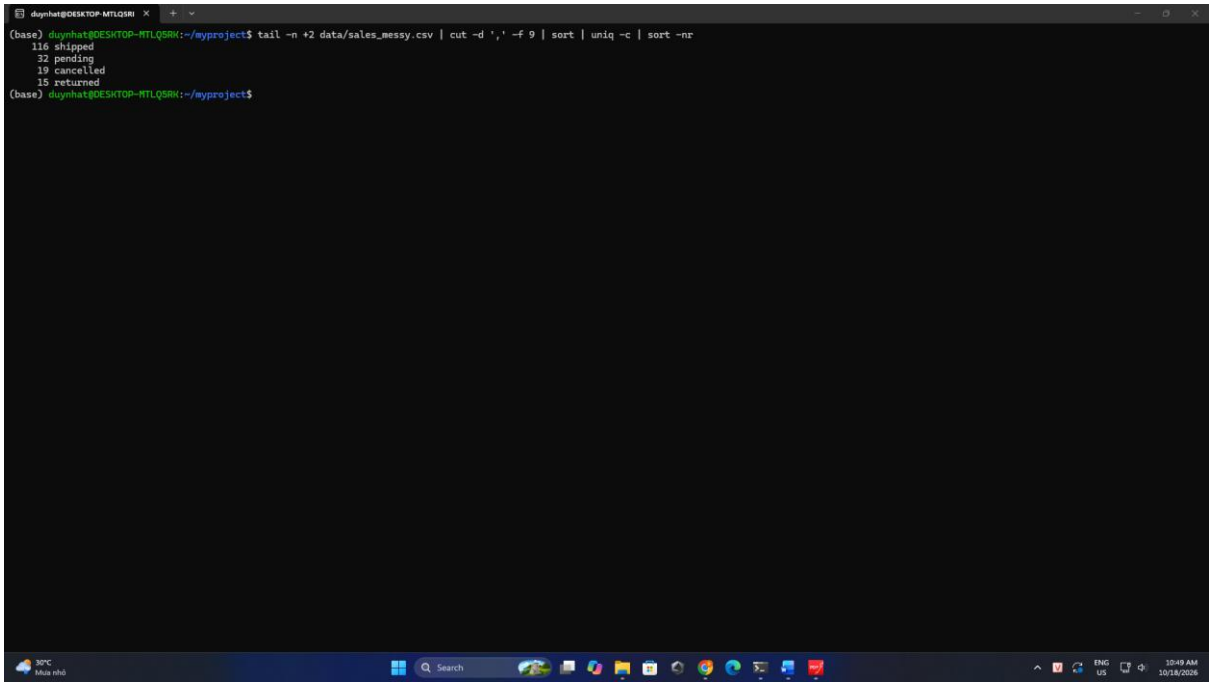
② Explanation of the syntax you used:

- The command ***cut -d ',' -f 9 data/sales_messy.cs***v is used to extract the ninth column (final column) status in the CSV file, using a comma as the delimiter.
- The command ***grep -x "cancelled"*** filters the output, keeping only lines that match the exact string “cancelled”.
- The command ***wc -l*** counts the remaining lines, corresponding to the number of cancelled orders. In sales_messy.csv, there are 19 cancelled orders.

Q7 · Orders per status, ranked

Produce a count of orders for each status, ranked from most to least frequent.

① Screenshot (paste the command and its output from your own terminal):



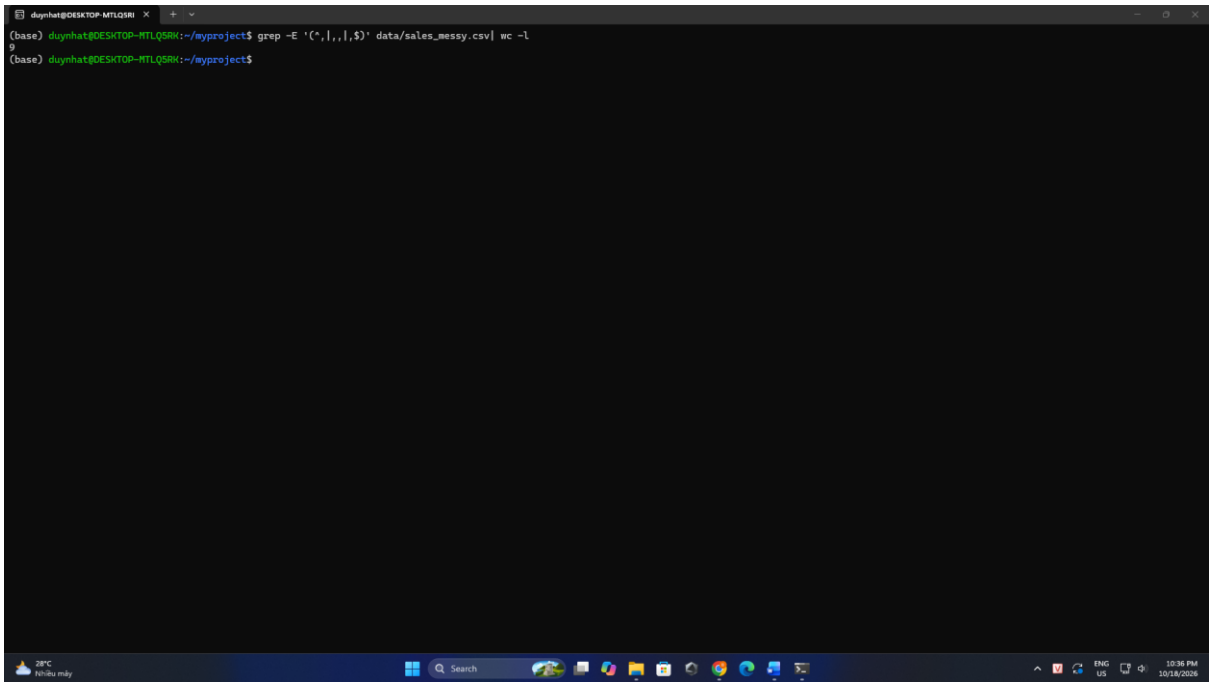
② Explanation of the syntax you used:

- The command ***tail -n +2 data/sales_messy.csv*** gets all lines from the second line to the end of the file, excluding the header.
- The command ***cut -d ',' -f 9 data/sales_messy.cs***v extracts the ninth column status (final column) in the remaining data.
- The ***sort*** command must be executed before ***uniq -c*** because it groups identical status values together, allowing ***uniq -c*** to count the occurrences of each status.
- Finally, the command ***sort -nr*** (-n: numerical, -r: reverse) sorts the output in descending numerical order based on the number of orders in each status.

Q8 · Find missing values

How many rows contain a missing field (an empty value that shows up as two commas in a row, ,,)? Explain how your command detects them.

① Screenshot (paste the command and its output from your own terminal):



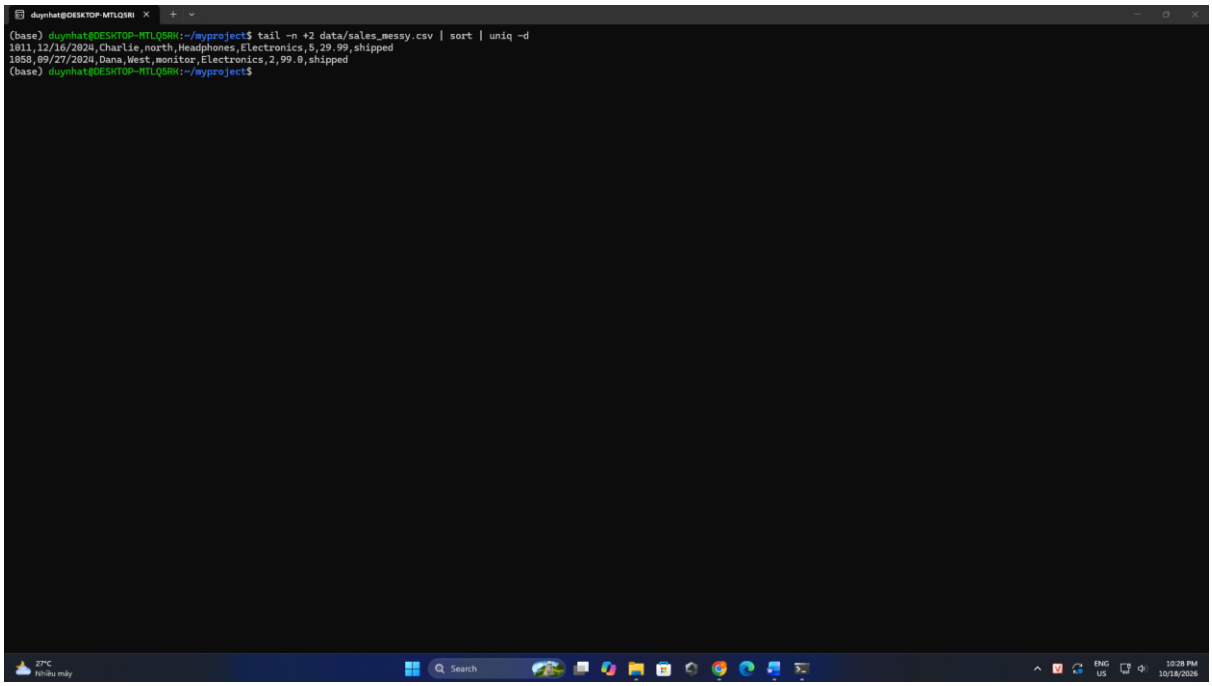
② Explanation of the syntax you used:

- The command ***grep -E '(^,|,,|,$)' data/sales_messy.csv*** filters the rows that contain an empty field. The flag -E enables extended regular expressions, so the alternation operator | and the group ( ) can be used without backslashes. The pattern has three alternatives: **^,** matches a comma at the beginning of a line (empty first field), **,,** matches two consecutive commas (empty field in the middle), and **,$** matches a comma at the end of a line (empty last field).
- ***wc*** with the ***-l*** flag counts the lines returned by the preceding command. In the sales_messy.csv, the number of rows containing a missing field is 9.

Q9 · Detect duplicate rows

Find the exact duplicate rows in the data. Explain why sorting is required before de-duplicating.

① Screenshot (paste the command and its output from your own terminal):



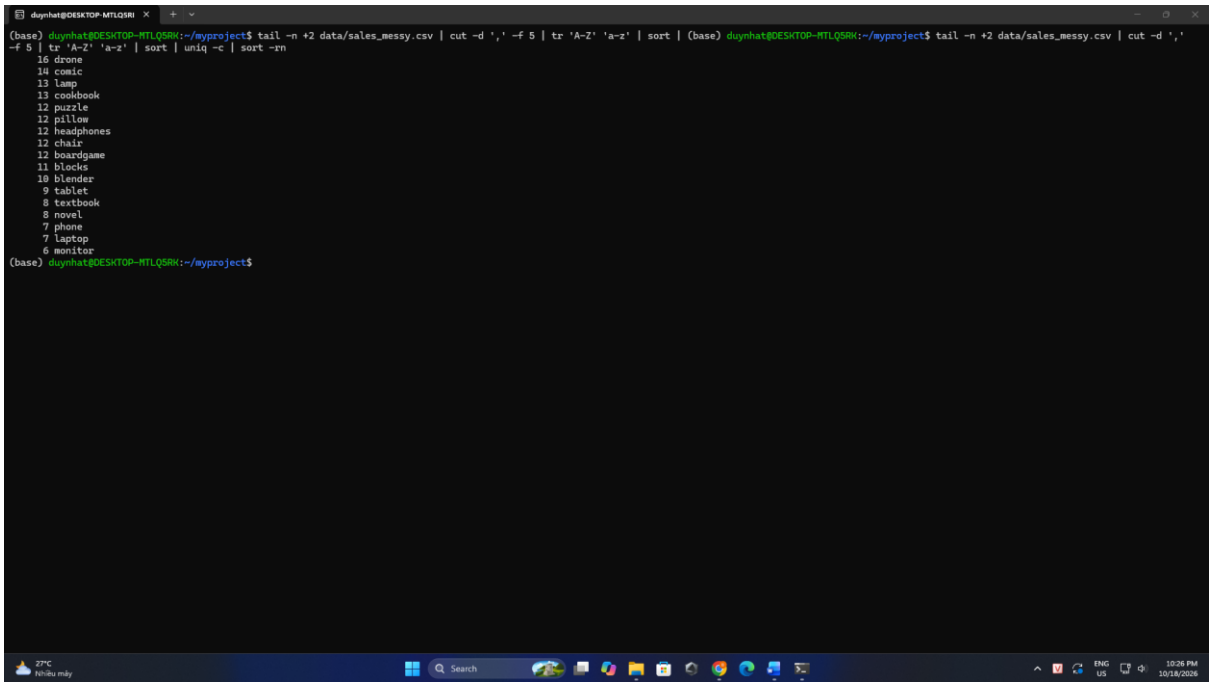
② Explanation of the syntax you used:

- The command ***tail -n +2 data/sales_messy.csv*** skips the header row. The ***sort*** command arranges the remaining rows so that identical rows appear consecutively. Then, ***uniq -d*** prints only the rows that occur more than once.
- ***sort*** is necessary because ***uniq*** only detects consecutive duplicate lines. Without sorting, identical rows located in different parts of the file might not be detected.

Q10 · Clean, then count

Count how many times each value appears in the product column (column 5). Which product is most common?

① Screenshot (paste the command and its output from your own terminal):



② Explanation of the syntax you used:

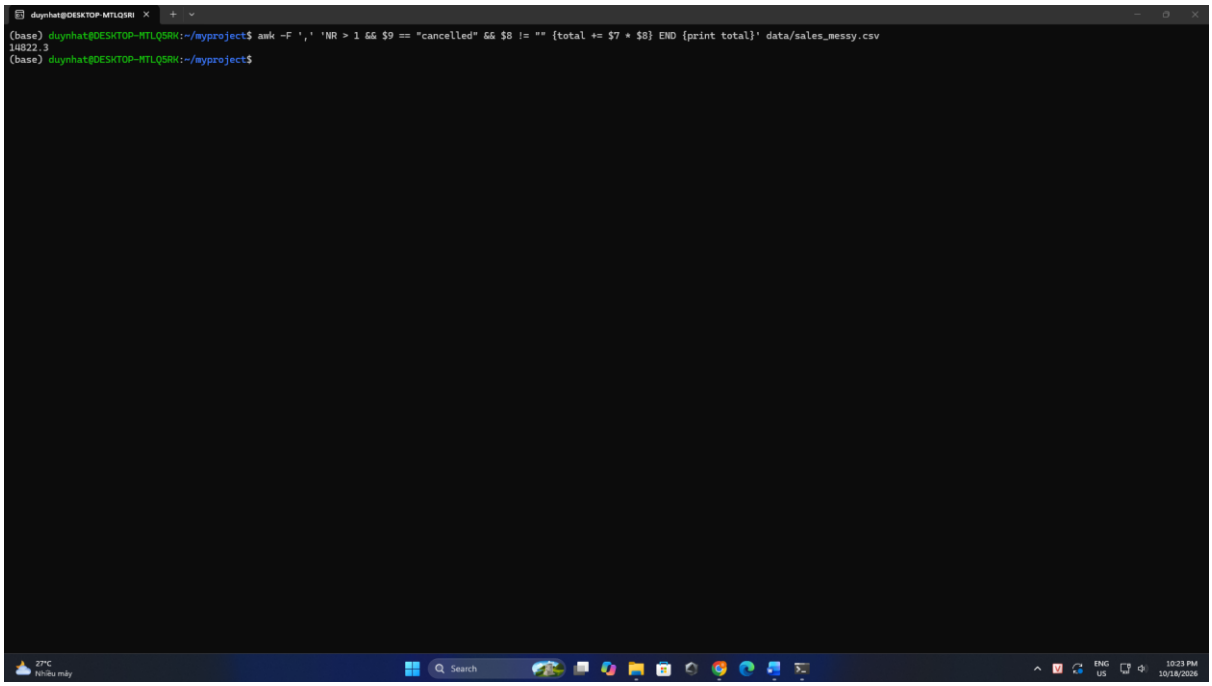
- The command ***tail -n +2 data/sales_messy.csv*** retrieves all lines from the second line to the end of the CSV file, excluding the header row.
- The command ***cut -d ',' -f 5*** extracts the fifth column, which contains the product names.
- The command ***tr 'A-Z ' 'a-z'*** converts all uppercase letters to lowercase, ensuring that product names differing only in capitalization are treated as the same product.
- The ***sort*** command must be executed before ***uniq -c*** because it places identical product names consecutively, allowing ***uniq -c*** to count the occurrences of each product.
- Finally, ***sort -nr*** sorts the products by their number of appearances in descending numerical order.

➔ The highest quantity is 16 and it belongs to ***drone*** which is the most common product.

Q11 · Revenue with awk

Using awk, compute the total revenue (quantity × unit_price) of all cancelled orders, skipping any row with a missing price. Explain the parts of your awk program.

① Screenshot (paste the command and its output from your own terminal):



② Explanation of the syntax you used:

The command ***awk -F ',' 'NR > 1 && $9 == "cancelled" && $8 != "" {total += $7 * $8} END {print total} ' data/sales_messy.csv***:

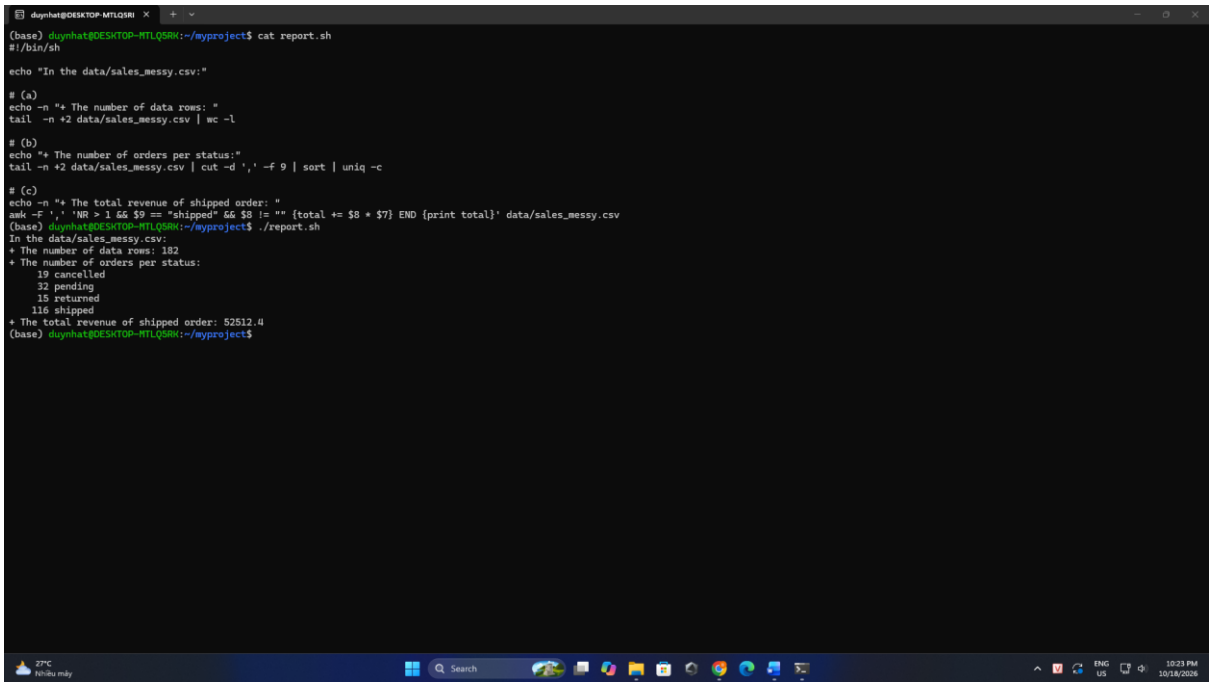
- The flag ***-F*** specifies the comma ‘,’ as the field separator.
- The condition ***NR > 1*** skips the header row.
- The condition ***$9 == "cancelled"*** selects only orders whose status is cancelled.
- The condition ***$8 != ""*** skips rows with a missing unit price.
- The statement ***total += $7 * $8*** calculates the revenue of each selected order by multiplying the quantity ($7) by the unit price ($8), then adds it to the running total.
- The ***END {print total}*** block prints the total revenue after all rows have been processed.
- The total revenue of cancelled order in sales_messy.csv is 14822.3

Part 4 — Scripting

Q12 · Write report.sh

Write a shell script report.sh that, for data/sales_messy.csv, prints: (a) the total number of data rows, (b) the count of orders per status, and (c) the total revenue of shipped orders. Screenshot both the script contents and its output when run. Explain, line by line, what your script does.

① Screenshot (paste the command and its output from your own terminal):



② Explanation of the syntax you used:

- ***#!/bin/sh:*** Specifies that the script should be executed using the /bin/sh shell.
- ***echo*** commands: Display messages and labels in the terminal to identify the results of each section. The echo -n option prints text without adding a trailing newline, allowing the output of the next command to appear on the same line.
- ***tail -n +2 data/sales_messy.csv | wc -l:*** Skips the header row using tail -n +2 and counts the remaining lines using wc -l, which provides the number of data rows in the CSV file.
- ***tail -n +2 data/sales_messy.csv | cut -d ',' -f 9 | sort | uniq -c:*** Skips the header, extracts the ninth column (status), sorts the values so identical statuses appear consecutively, and counts each status using uniq -c, which provides the number of orders per status.
- ***awk -F ',' 'NR > 1 && $9 == "shipped" && $8 != "" {total += $8 * $7} END {print total}' data/sales_messy.csv:*** Uses awk with a comma as the field separator. It skips the header (NR > 1), selects rows whose status in column 9 is shipped, skips rows with an empty unit price in column 8, multiplies the unit price ($8) by the quantity ($7), accumulates the revenue in total, and prints the total after processing all rows.